

Operating Systems and Networks

[Mini Projects](#) / Mini Project 1

Before You Start

Deadline

The deadline for this assignment is **Thursday, September 11, 2025 23:59:59 IST**.

Grading

Marks are divided into 2 categories

- **I/O Evaluation**
 - This component will be **automatically evaluated**.
 - Your shell will be tested on a common testing environment.
 - This will be done in a similar manner to how OJ worked in CPro and DSA.
- **Code Evaluation**
 - This component will be **manually evaluated**.
 - Your code and understanding will be evaluated by TAs during an in-person viva.

If you are using LLMs for any portion of the code,

- Enclose the LLM generated portion as below,

```
##### LLM Generated Code Begins #####  
some llm generated code  
##### LLM Generated Code Ends #####
```

- Provide the exact prompts and LLM responses as image files in a folder called `llm_completions/`

Notes Regarding Grading

- In the in-person code evaluation, you will be tested **solely on the understanding of your code**.
- The exact weightage will be released shortly, but note that the in-person code evaluation **will hold more weightage** than the automated evaluation.

Since evaluation of the correctness of your code will be automatically evaluated, **you must ensure that your shell strictly follows the input and output requirements mentioned in this document.**

Failure to follow input/output formats will likely lead to **a score of zero** in the automatic evaluation. No queries will be entertained.

A sample testing script will be released shortly that will contain sample test cases that you may use to check whether your shell follows the specified input/output format. Note that these test cases are **a small subset of the ones used in the automatic evaluation.**

Notes Regarding Queries

Before asking a doubt please thoroughly read through the requirements.

If a doubt is already answered by the requirements, it **will not be answered.**

You may only ask doubts regarding input/output formats.

For all other doubts, **make reasonable assumptions.**

C Shell [Total: 645]

General Requirements

The project must be broken down into multiple .c and .h files based on functionality.

Monolithic code in a single file will be heavily penalized.

You may only use the C POSIX library headers and functions. The documentation for these are provided here - <https://pubs.opengroup.org/onlinepubs/9699919799/idx/head.html>.

Use the below gcc feature flags while compiling to ensure POSIX compliance.

```
gcc -std=c99 \  
-D_POSIX_C_SOURCE=200809L \  
-D_XOPEN_SOURCE=700 \  
-Wall -Wextra -Werror \  
-Wno-unused-parameter \  
-fno-asm \  
your_file.c
```

Your final code submission **must** be compilable using the command `make all` in the **shell directory** of the git repository. It **must** compile the shell to the file `shell.out`. If

not done, this would cause automatic evaluation to fail, leading to zero marks. ~A test script will be provided soon~ (Done, see Moodle). This binary should be created in the **shell directory**, and not the project root.

Part A: Shell Input [Total: 65]

This is the base for the rest of the assignment. Work hard on this!

Banned Syscalls for Part A: `exec*` , i.e. any of the syscalls whose names start with `exec`.

A.1: The Shell Prompt [10]

Your shell should show a prompt so that the user knows that they can provide input to it. Successfully completing most of this requirement merely requires your shell to display the below prompt!

```
<Username@SystemName:current_path>
```

Find out what `Username` and `SystemName` are by investigating the `bash` (or `zsh`, etc.) prompt that you see in your computer's shell. Unix only, sorry non-WSL Windows users :(

REQUIREMENTS

- 1 The shell should display the above prompt when it is not running a foreground process.
- 2 The directory in which the shell is started becomes the shell's home directory.
- 3 When the current working directory has the home directory as an ancestor (for example `/path/to/home/meow`), the absolute path to the home directory must be replaced by a tilde `"~"`. So the example would be displayed as `~/meow` .
- 4 When the current working directory does not have the home directory as an ancestor (for example `/path/to/not_home/meow`) the absolute path should be displayed as is. So the example would be displayed as `/path/to/not_home/meow` .

EXAMPLE

```
# I am currently running bash
[rudy@iit ~/osnmp1/]$ make
[rudy@iit ~/osnmp1/]$ ./shell.out
# I am now running my shell
<rudy@iit:~>
```

A.2: User Input [5]

A shell's job is to take input from a user, parse it, and then run any commands if necessary. For now, after taking input from a user, the shell doesn't have to do anything with the input!

So successfully completing this requirement merely requires your shell to be able to take user input!

REQUIREMENTS

- 1 The shell should allow a user to type input.
- 2 When the user presses the enter/return key, the shell should consume the input.
- 3 After consuming the input, the shell should once again display the shell prompt. The user should again be able to type input and submit it (by pressing the enter/return key).

EXAMPLE

```
<rudy@iiiit:~> Hi there guys!
<rudy@iiiit:~> This shell is cool!
<rudy@iiiit:~>
```

A.3: Input Parsing [50]

After taking user input, the input must be parsed so that we can decide what the user wants the shell to do. You will be implementing a parser for the below Context Free Grammar

```
shell_cmd  -> cmd_group ((& | ;) cmd_group)* &?
cmd_group -> atomic (\| atomic)*
atomic    -> name (name | input | output)*
input     -> < name | <name
output    -> > name | >name | >> name | >>name
name      -> r"^[^&><;\s]+"
```

Below is another version of the **same grammar**, but provided in a pure regex format. This is to clarify that in the above grammar, **anywhere there is a space, you must handle an arbitrary amount of whitespace**. Note the usage of '*' and '+'.

```
shell_cmd  -> r" (?P<cmd_group1>.+)(?P<WS1>\s*)((&|;)(?P<WS2>\s*)(?P<cmd_group
cmd_group -> r" (?P<atomic1>.+)(?P<WS1>\s*)(\|(?P<WS2>\s*)(?P<atomic2>.+))*"
atomic    -> r" (?P<NAME1>^[^&><;\s]+)(?P<WS1>\s+)((?P<NAME2>^[^&><;\s]+)|(?P<input
input     -> r"<(?P<WS>\s*)(?P<NAME>^[^&><;\s]+)"
output    -> r">|>>)(?P<WS>\s*)(?P<NAME>^[^&><;\s]+)"
```

Instead of attempting to understand any of these regexes, I highly recommend copying the regex (the text between the double quotes) and pasting it into [Regex101](https://regex101.com/). Ensure that you select the flavor to be 'Python' and that you disable the 'global' and 'multiline' options which

are auto enabled in the 'Regex Flags' menu. An explanation for the provided regex can be seen on the right hand side.

A simple introduction to Context Free Grammars and an example parser for a simpler grammar will be provided in the tutorial.

Parsers for such programs usually create something called an Abstract Syntax Tree. However, you are not required to parse the input into an AST. You can use any structure that you find the most convenient!

REQUIREMENTS

- 1 The shell should verify whether or not an inputted command is valid or invalid using the rules of the grammar.
- 2 If a command is valid, do nothing (for now)! For example, `cat meow.txt | meow; meow > meow.txt &` is a valid command.
- 3 If a command is invalid, print "Invalid Syntax!". For example, `cat meow.txt | ; meow` is an invalid command.
- 4 Your parser should ignore whitespace characters (space, tab (`\t`), new line (`\n`), and carriage return (`\r`)) in between valid tokens.

EXAMPLE

```
# This is valid syntax
<rudy@iiiit:~> Hi there guys!
# This isn't
<rudy@iiiit:~> cat meow.txt | ; meow
Invalid Syntax!
<rudy@iiiit:~>
```

Part B: Shell Intrinsics [Total: 70]

These are commands that any shell worth its salt supports! Implementing them shouldn't be too difficult.

Banned Syscalls for Part B: `exec*`, i.e. any of the syscalls whose names start with `exec`.

B.1: hop [20]

Syntax: `hop ((~ | . | .. | - | name)*)?`

Purpose: The hop command allows a user to change the shell's current working directory.

REQUIREMENTS

Execute one of the following operations sequentially for each passed argument:

- 1 "~" or No Arguments: Change the CWD to the shell's home directory.
- 2 ".": Do nothing (i.e. stay in the same CWD)
- 3 "..": Change the CWD to the parent directory of the CWD, or do nothing if the CWD has no parent directory.
- 4 "-": Change the CWD to the previous CWD or do nothing if there was no previous CWD.
So, after starting the shell, till the first hop command which was not a "-" was run, you must do nothing.
- 5 "name": Change the CWD to the specified relative or absolute path.
- 6 If the directory does not exist, output "No such directory!"

EXAMPLE

```
<rudu@iiiit:~/osnmp1> hop ~
<rudu@iiiit:~> hop ..
<rudu@iiiit:/home/> hop rudy/osnmp1 .. -
<rudu@iiiit:~/osnmp1>
```

B.2: reveal [20]

Syntax: reveal `(-(a | l)*)* (~ | . | .. | - | name)?`

Purpose: The reveal command allows a user to view the files and directories in a directory.

REQUIREMENTS

Flags modify the default behavior of reveal.

- 1 "a": When this flag is set, reveal all files and directories, including hidden ones (files and directories starting with .). The default behavior is to not reveal hidden files and directories.
- 2 "l": When this flag is set, reveal files and directories in a line by line format, i.e. only one entry per line.
- 3 When both flags are set, all files and directories, including hidden ones, must be printed in the line by line format.
- 4 When neither flag is set, print files and directories in the format of `ls`.
- 5 The argument passed invokes identical behavior to hop, except that here we are listing directory contents instead of changing the CWD.
- 6 Ensure that the files are always listed in lexicographic order. Note that you are **not** required to implement the format of `ls -l`. (In fact if you do you may lose marks due to the automated evaluation!) Further note that here you must use the ASCII values of the

characters to sort the names lexicographically.

- 7 If the directory does not exist, output “No such directory!”
- 8 If reveal is passed too many arguments, output “reveal: Invalid Syntax!”
- 9 If `reveal -` is run before any `hop` commands have been run since the start of the shell, output “No such directory!”

EXAMPLE

```
<rudy@iiiit:~> reveal ~
osnmp1
<rudy@iiiit:~> hop ..
<rudy@iiiit:/home> reveal
rudy
<rudy@iiiit:/home> hop rudy/osnmp1
<rudy@iiiit:~/osnmp1> reveal -la
.git
.gitignore
Makefile
README.md
include
llm_completions
src
shell.out
<rudy@iiiit:~/osnmp1> reveal -lalalalaaaalal -lalala -al
.git
.gitignore
Makefile
README.md
include
llm_completions
src
shell.out
<rudy@iiiit:~/osnmp1> reveal -aaaaaaa -a
.git .gitignore include llm_completions src shell.out Makefile README.md
```

B.3: log [30]

Syntax: `log (purge | execute <index>)?`

Purpose: The `log` command allows a user to view their recently executed commands.

REQUIREMENTS

- 1 The stored list of commands must persist across shell sessions.
- 2 Store a maximum of 15 commands. Overwrite the oldest command.
- 3 Do not store a command if it is identical to the previously executed command in the log. Here identical can mean syntactically or exactly. Take it to mean exactly.
- 4 Always store the entire `shell_cmd` as defined in the CFG.
- 5 Do not store any `shell_cmd` if the command name of an atomic command is `log` itself.
- 6 The command exhibits three behaviors:
 - No arguments: Print the stored commands in order of oldest to newest.
 - `purge` : Clear the history.
 - `execute <index>` : Execute the command at the given index (one-indexed, indexed in order of newest to oldest). Do not store the executed command.
- 7 If the syntax is incorrect (Ex: `log log`), print "log: Invalid Syntax!"

EDIT - I recommend implementing this command after implementing part C.1.

EXAMPLE

```
<rudy@iiiit:~> reveal ~
osnmp1
<rudy@iiiit:~> hop ..
<rudy@iiiit:/home/rudy> reveal
osnmp1
<rudy@iiiit:/home/rudy> hop
<rudy@iiiit:~> log
reveal ~
hop ..
reveal
hop
<rudy@iiiit:~> log execute 2
osnmp1
<rudy@iiiit:~> log
reveal ~
hop ..
reveal
hop
<rudy@iiiit:~> log purge
<rudy@iiiit:~> log
<rudy@iiiit:~>
```


Part C: File Redirection and Pipes [Total: 200]

For this part, you will implement I/O redirection and command piping. When processing commands with sequential (`;`) or background (`&` , `&&`) operators, you should only execute the first `cmd_group` and ignore the rest for now.

C.1: Command Execution

This part was implicitly required, and has just been added explicitly for clarity. You must allow the execution of **arbitrary commands**. This includes commands like `cat` , `echo` , `sleep` , etc. If a command does not exist (example `dosakdaoskdos`), output `Command not found!` .

C.2: Input Redirection [50]

Syntax: `command < filename`

Purpose: The input redirection operator allows a command to read its standard input from a file instead of the terminal.

REQUIREMENTS

- 1 The shell must open the specified file for reading using the `open()` system call with `O_RDONLY` flag.
- 2 If the file does not exist or cannot be opened, the shell must print “No such file or directory” and not execute the command.
- 3 The shell must redirect the command’s standard input (`STDIN_FILENO`) to the opened file using `dup2()` .
- 4 The shell must close the original file descriptor after duplication to avoid file descriptor leaks.
- 5 When multiple input redirections are present (e.g., `command < file1 < file2`), only the last one must take effect.

C.3: Output Redirection [50]

Syntax: `command > filename` or `command >> filename`

Purpose: The output redirection operators allow a command to write its standard output to a file instead of the terminal.

REQUIREMENTS

- 1 For `>` , the shell must create a new file (wipe it if it already exists) and open it for writing.
- 2 For `>>` , the shell must append to the passed file (or create if it doesn’t exist) and open it for appending.
- 3 When multiple output redirections are present (e.g., `command > file1 > file2`), only

the last one must take effect.

- 4 Input and output redirection must work together (e.g., `command < input.txt > output.txt`).
- 5 If the output file cannot be created for some reason, the shell must print “Unable to create file for writing” and not execute the command.

C.4: Command Piping [100]

Syntax: `command1 | command2 | ... | commandN`

Purpose: The pipe operator allows the standard output of one command to be connected to the standard input of the next command.

REQUIREMENTS

- 1 The shell must create pipes using the `pipe()` system call for each `|` operator in the command.
- 2 For each command in the pipeline, the shell must fork a child process.
- 3 The shell must redirect the standard output of `command[i]` to the write end of `pipe[i]` .
- 4 The shell must redirect the standard input of `command[i+1]` to the read end of `pipe[i]` .
- 5 The parent shell must wait for all commands in the pipeline to complete.
- 6 A piped command sequence is considered finished only when all processes in the pipeline have exited.
- 7 If any command in the pipeline fails to execute, the pipeline must still attempt to run the remaining commands.
- 8 File redirection and pipes must work together (e.g., `command1 < input.txt | command2 > output.txt`).

Part D: Sequential and Background Execution [Total: 200]

D.1: Sequential Execution [100]

Syntax: `command1 ; command2 ; ... ; commandN`

Purpose: The semicolon operator allows multiple commands to be executed one after another.

REQUIREMENTS

- 1 The shell must execute each command in the order they appear.
- 2 The shell must wait for each command to complete before starting the next.

- 3 If a command fails to execute, the shell must continue executing the subsequent commands.
- 4 Each command in the sequence must be treated as a complete `shell_cmd` as defined in the grammar.
- 5 The shell prompt must only be displayed after all commands in the sequence have finished executing.

D.2: Background Execution [100]

Syntax: `command &`

Purpose: The ampersand operator allows a command to run in the background while the shell continues to accept new commands.

REQUIREMENTS

- 1 When a command ends with `&`, the shell must fork a child process but not wait for it to complete.
- 2 The shell must print the background job number and process ID in the format:
`[job_number] process_id`
- 3 The shell must immediately display a new prompt after launching the background process.
- 4 After an user inputs, before parsing the input, the shell must check for completed background processes.
- 5 When a background process completes successfully, the shell must print: `command_name with pid process_id exited normally`
- 6 When a background process exits abnormally, the shell must print: `command_name with pid process_id exited abnormally`
- 7 Background processes must not have access to the terminal for input.

Part E: Exotic Shell Intrinsics [Total: 110]

E.1: activities [20]

Syntax: `activities`

Purpose: The `activities` command lists all processes spawned by the shell that are still running or stopped.

REQUIREMENTS

- 1 The command must display each process in the format: `[pid] : command_name - State`

- 2 The command must sort the output lexicographically by command name before printing.
- 3 The command must remove processes from the list once they have terminated.
- 4 Running processes must show state as “Running” and stopped processes as “Stopped”.

E.2: ping [20]

Syntax: ping <pid> <signal_number>

Purpose: The ping command sends a signal to a process with the specified PID.

REQUIREMENTS

- 1 The command must take the signal number modulo 32 before sending: $\text{actual_signal} = \text{signal_number} \% 32$
- 2 If the process does not exist, the command must print “No such process found”
- 3 On successful signal delivery, the command must print “Sent signal signal_number to process with pid <pid> ”
- 4 If the inputted signal_number is not a valid number, print Invalid syntax! .

E.3: Ctrl-C, Ctrl-D and Ctrl-Z [30]

Purpose: These keyboard shortcuts provide job control functionality.

REQUIREMENTS FOR CTRL-C (SIGINT)

- 1 The shell must install a signal handler for SIGINT.
- 2 The handler must send SIGINT to the current foreground child process group if one exists.
- 3 The shell itself must not terminate on Ctrl-C.

REQUIREMENTS FOR CTRL-D (EOF)

- 1 The shell must detect the EOF condition.
- 2 The shell must send SIGKILL to all child processes.
- 3 The shell must exit with status 0.
- 4 The shell must print “logout” before exiting.

REQUIREMENTS FOR CTRL-Z (SIGTSTP)

- 1 The shell must install a signal handler for SIGTSTP.
- 2 The handler must send SIGTSTP to the current foreground child process group if one exists.
- 3 The shell must move the stopped process to the background process list with status “Stopped”.
- 4 The shell must print: [job_number] Stopped command_name
- 5 The shell itself must not stop on Ctrl-Z.

E.4: fg and bg [40]

Syntax: fg [job_number] and bg [job_number]

Purpose: The fg and bg commands control background and stopped jobs.

REQUIREMENTS FOR FG COMMAND

- 1 The command must bring a background or stopped job to the foreground.
- 2 If the job is stopped, the command must send SIGCONT to resume it.
- 3 The shell must wait for the job to complete or stop again.
- 4 If no job number is provided, the command must use the most recently created background/stopped job.
- 5 If the job number doesn't exist, the command must print "No such job"
- 6 The command must print the entire command when bringing it to foreground.

REQUIREMENTS FOR BG COMMAND

- 1 The command must resume a stopped background job by sending SIGCONT.
- 2 The job must continue running in the background after receiving the signal.
- 3 The command must print [job_number] command_name & when resuming.
- 4 If the job is already running, the command must print "Job already running"
- 5 If the job number doesn't exist, the command must print "No such job"
- 6 Only stopped jobs can be resumed with bg; running jobs must produce "Job already running"

Networking [Total 80]

Objective: In this part of the mini-project you will be building upon the the unreliable UDP protocol to improve upon its reliability and you will be simulating several core TCP functionalities from scratch. This section is designed to make you familiar with socket programming, TCP and UDP protocols, supporting both file transfer and real-time chat.

1. Core Functionalities

1.1 S.H.A.M. PACKET STRUCTURE [5]

All communication must occur via UDP datagrams. The payload of each datagram will be a **S.H.A.M.** packet, which you must define using a `struct`. This header must precede any application data.

// Recommended S.H.A.M. Header Structure

```
struct sham_header {  
  
    uint32_t seq_num; // Sequence Number  
  
    uint32_t ack_num; // Acknowledgment Number  
  
    uint16_t flags; // Control flags (SYN, ACK, FIN)  
  
    uint16_t window_size; // Flow control window size  
  
};
```

Sequence Number (seq_num): A 32-bit field indicating the byte-stream number of the first byte in this packet's data segment.

Acknowledgment Number (ack_num): A 32-bit field that contains the value of the next sequence number the sender of the ACK is expecting to receive. This will be a **cumulative acknowledgment**.

Flags (flags): A 16-bit field for connection management. You must implement:

SYN (Synchronise): 0x1 - Used to initiate a connection.

ACK (Acknowledge): 0x2 - Indicates the ack_num field is significant.

FIN (Finish): 0x4 - Used to terminate a connection.

Window Size (window_size): A 16-bit field for flow control. It specifies the number of data bytes the sender of this packet is willing to accept.

1.2 CONNECTION MANAGEMENT [10]

Your protocol must establish and terminate connections gracefully.

Three-Way Handshake (Establishment):

- 1 **Client -> Server:** Sends a packet with the SYN flag set and an initial sequence number X .
- 2 **Server -> Client:** Responds with a packet where both SYN and ACK flags are set. It uses its own initial sequence number Y and acknowledges the client with $\text{ack_num} = X + 1$.
- 3 **Client -> Server:** Completes the handshake by sending a packet with the ACK flag set

and `ack_num = Y + 1` . The connection is now established.

Four-Way Handshake (Termination):

- 1 **Initiator -> Other Side:** Sends a `FIN` packet.
- 2 **Other Side -> Initiator:** Responds with an `ACK` .
- 3 **Other Side -> Initiator:** Once it's ready to close, sends its own `FIN` .
- 4 **Initiator -> Other Side:** Responds with a final `ACK` .

1.3 DATA SEQUENCING AND RETRANSMISSION[25]

Data Segmentation: The sender must read a file (or user input) and break it into fixed-size chunks (1024 bytes). Each chunk is the payload for a **S.H.A.M.** packet.

Sliding Window: The sender can transmit multiple packets without waiting for an acknowledgment for each one. The number of unacknowledged packets in flight must not exceed a fixed window size (this is different from the `window_size` and is fixed e.g., 10 packets).

Cumulative ACKs: The receiver should send an `ACK` for the highest in-order sequence number received. For example, if packets 1, 2, and 4 arrive, the receiver sends `ACK` for sequence 3 (since it's expecting packet 3 next). It should buffer packet 4.

Retransmission Timeout (RTO): The sender must maintain a timer for each packet sent. If an `ACK` for a given packet is not received within a timeout period (e.g., 500ms), the packet must be retransmitted.

Example 1: Basic Retransmission Scenario

This example demonstrates how the system recovers when a single packet is lost.

Scenario Assumptions:

Sender's Sliding Window: 4 packets

Packet Data Size: 1024 bytes

Sequence Numbers (SEQ): Byte-based

Step 1: Initial Transmission

The sender transmits a window of four packets and starts a timer for each one.

`SND DATA SEQ=1`

SND DATA SEQ=1025 ← **This packet is lost**

SND DATA SEQ=2049

SND DATA SEQ=3073

Step 2: Receiver's Cumulative ACK

The receiver gets packets 1, 3, and 4. Since packet 2 (SEQ=1025) is missing, it can only acknowledge the data it has received contiguously. It buffers packets 3 and 4.

It sends an ACK for the next byte it expects: RCV sends: ACK=1025

Step 3: Timeout and Selective Retransmission 🕒

The sender's timer for packet 2 (SEQ=1025) expires. It assumes the packet was lost and retransmits **only that packet**.

TIMEOUT SEQ=1025

RETX DATA SEQ=1025

Step 4: Recovery and Final ACK

The receiver gets the retransmitted packet 2. It can now process its buffered packets (3 and 4), completing the sequence up to byte 4096.

It sends a new cumulative ACK for the entire block: RCV sends: ACK=4097

Example 2: The Efficiency of Cumulative ACKs

This section addresses a common question about the previous scenario.

Question: After retransmitting packet 2, does the sender also need to retransmit packets 3 and 4?

Answer: No. The sender does not retransmit packets 3 and 4.

Explanation: The final ACK=4097 sent by the receiver is a **cumulative acknowledgment**. This single message efficiently informs the sender of two things:

- 1 "I have successfully received **all data** up to and including byte 4096."
- 2 "I am now ready for the next piece of data, which starts at byte 4097."

When the sender receives this ACK, it knows that the retransmitted packet 2, as well as the originally sent packets 3 and 4, have all been successfully received. It can then cancel any running timers for those packets and continue sending new data. This prevents unnecessary

retransmissions and makes the protocol highly efficient.

2. Flow Control[10]

You must implement a basic sliding window flow control mechanism.

The receiver must always include its current available buffer space (in bytes) in the `window_size` field of every packet it sends.

The sender must read this `window_size` from incoming `ACK` packets.

The sender must ensure that the amount of unacknowledged data it has in flight (`LastByteSent - LastByteAcked`) is **always less than or equal to** the receiver's advertised `window_size`.

3. Implementation and Testing Requirements

To enable quick and automated testing, your programs **must** adhere to the following specifications.

3.1 COMMAND-LINE INTERFACE

Your client and server must be executable with the following arguments, supporting two modes of operation.

Server:

```
./server <port> [--chat] [loss_rate]
```

Client:

File Transfer Mode (Default)

```
./client <server_ip> <server_port> <input_file> <output_file_name> [loss_rate]
```

Chat Mode

```
./client <server_ip> <server_port> --chat [loss_rate]
```

`--chat` : An optional flag to activate Chat Mode. When used, all file-related arguments are ignored.

`[loss_rate]` : An optional floating-point value between 0.0 and 1.0 indicating the packet loss probability. If not provided, it defaults to 0.0.

3.2 MODE-SPECIFIC BEHAVIOR [10]

File Transfer Mode (Default): After the handshake, the client sends the specified file to the server. The server receives and saves the file. This is the behavior if the `--chat` flag is **not** present.

Chat Mode (`--chat`): After the handshake, both client and server enter a loop to handle concurrent input from the keyboard (`stdin`) and the network socket. Typing `/quit` in the chat should initiate the 4-way `FIN` handshake to terminate the connection.

Hint: Use the `select()` system call to monitor both `stdin` (file descriptor 0) and your socket to handle I/O without threads.

3.3 STANDARDIZED OUTPUT FOR VERIFICATION [5]

This requirement applies to File Transfer Mode only. Upon successfully receiving the entire file and closing the connection, the server **must** perform two actions:

- 1 Calculate the MD5 checksum of the received file.
- 2 Print the result to `stdout` in the following exact format:

```
MD5: <32-character_lowercase_md5_hash>
```

Linux

Step1: Install the library

- 1 In your terminal, run the following command to update your package lists and install the required package (`libssl-dev`):

```
sudo apt update && sudo apt install libssl-dev
```

Step 2: Compile Your Code Correctly

Once the library is installed, you can compile your code. You **do not** need the special `-I` and `-L` flags that were required for macOS.

- 1 **Use this simple command template** to compile your program. Replace `your_program.c` and `-o your_program` with your actual filenames.

```
gcc your_program.c -o your_program -lcrypto
```

MacOS

Step 1: Install the OpenSSL Library

Now, use Homebrew to install the OpenSSL library. It's a simple, one-line command.

- 1 In your Terminal, run:

```
brew install openssl
```

Step 2: Compile Your Code Correctly

Because Homebrew installs OpenSSL in a special location, you can't just use a simple `gcc` command. You must include special flags to tell the compiler where to find the OpenSSL files.

- 1 **Use this specific command template** to compile your program. Replace `your_program.c` and `-o your_program` with your actual filenames.

```
gcc your_program.c -o your_program -I$(brew --prefix openssl)/include -L$(b
```

3.4 SIMULATING PACKET LOSS FOR TESTING [5]

This requirement applies to both modes. The receiver must programmatically drop incoming data packets based on the optional `loss_rate` command-line argument to test retransmission logic.

4. Verbose Logging for Evaluation [10]

To allow the evaluator to verify the internal mechanics of your protocol, you must implement a verbose logging mode that writes metadata about key events, prefixed with a high-precision timestamp, to a dedicated log file.

Activation: The logging mode **must** be activated by setting an environment variable `RUDP_LOG=1`. If this variable is not set, no log file should be created or written to.

Log File Naming: The log file must be named according to the program's role:

Server: `server_log.txt`

Client: `client_log.txt`

Log Line Format: Each line in the log file must start with a timestamp in `[YYYY-MM-DD HH:MM:SS.microseconds]` format, followed by the `[LOG]` prefix and the event description.

IMPLEMENTATION NOTE

To get microsecond-level precision for your timestamps, you cannot use the standard `time()` function. You should use `gettimeofday()` from `<sys/time.h>`.

```
#include <stdio.h>
```

```
#include <sys/time.h>
```

```
#include <time.h>

// Inside your logging function

char time_buffer[30];
struct timeval tv;
time_t curtime;

gettimeofday(&tv, NULL);
curtime = tv.tv_sec;

// Format the time part
strftime(time_buffer, 30, "%Y-%m-%d %H:%M:%S", localtime(&curtime));

// Add microseconds and print to the log file
fprintf(log_file, "[%s.%06ld] [LOG] Your message here\n", time_buffer, tv.tv_usec);
```

REQUIRED LOG EVENTS

Your implementation must write a timestamped log message to the designated log file for each of the following events when logging is active:

1 Connection Handshake:

```
[timestamp] [LOG] SND SYN SEQ=<num>

[timestamp] [LOG] RCV SYN SEQ=<num>

[timestamp] [LOG] SND SYN-ACK SEQ=<num> ACK=<num>

[timestamp] [LOG] RCV ACK FOR SYN
```

1 Data Transmission:

```
[timestamp] [LOG] SND DATA SEQ=<num> LEN=<bytes>

[timestamp] [LOG] RCV DATA SEQ=<num> LEN=<bytes>
```

1 Acknowledgments:

```
[timestamp] [LOG] SND ACK=<num> WIN=<window_size>

[timestamp] [LOG] RCV ACK=<num>
```

1 Retransmission:

```
[timestamp] [LOG] TIMEOUT SEQ=<num>
```

```
[timestamp] [LOG] RETX DATA SEQ=<num> LEN=<bytes>
```

1 Flow Control:

```
[timestamp] [LOG] FLOW WIN UPDATE=<new_window_size>
```

1 Simulated Packet Loss:

```
[timestamp] [LOG] DROP DATA SEQ=<num>
```

EXAMPLE LOG FILE CONTENT (`server_log.txt`)

Here is a sample of what we expect to see inside the `server_log.txt` file. The program's standard output (`stdout`) should still *only* contain the final MD5: line.

```
[2025-08-03 17:38:15.123456] [LOG] RCV SYN SEQ=100
```

```
[2025-08-03 17:38:15.123589] [LOG] SND SYN-ACK SEQ=5000 ACK=101
```

```
[2025-08-03 17:38:15.124821] [LOG] RCV ACK FOR SYN
```

```
[2025-08-03 17:38:15.125111] [LOG] RCV DATA SEQ=101 LEN=1024
```

```
[2025-08-03 17:38:15.125185] [LOG] SND ACK=1125 WIN=8192
```

```
[2025-08-03 17:38:15.626234] [LOG] TIMEOUT SEQ=1125
```

```
[2025-08-03 17:38:15.626301] [LOG] RETX DATA SEQ=1125 LEN=1024
```

```
[2025-08-03 17:38:15.628991] [LOG] RCV ACK=2149
```

```
[2025-08-03 17:38:16.010101] [LOG] RCV FIN SEQ=4197
```

```
[2025--08-03 17:38:16.010182] [LOG] SND ACK FOR FIN
```

```
[2025-08-03 17:38:16.010250] [LOG] SND FIN SEQ=8500
```

```
[2025-08-03 17:38:16.011500] [LOG] RCV ACK=8501
```

xv6 [Total Marks - 140]

Please attend the tutorial on xv6 to understand how to set up and get started with xv6.

Part A - Basic System Call: getreadcount

Implement a system call that tracks and returns the total number of bytes read by the `read()` system call across all processes since boot.

Requirements

A.1: SYSTEM CALL IMPLEMENTATION

- 1 Implement `sys_getreadcount()` that returns the current number of bytes read by the `read()` syscall.
- 2 Handle overflow by wrapping around to 0.

A.2: USER PROGRAM

Create a user program `readcount.c` that:

- 1 Calls `getreadcount()` and prints the initial value
- 2 Reads 100 bytes from a file
- 3 Calls `getreadcount()` again and verifies the increase

Part B - Completely Fair Scheduler

You need to implement a simplified version of the Completely Fair Scheduler (CFS) in xv6. The CFS is designed to provide fair CPU time distribution among processes.

Background

The default scheduling policy in xv6 is round-robin-based. In this task, you'll implement two other scheduling policies and incorporate them in xv6. The kernel should only use one scheduling policy declared at compile time, with a default of round robin in case none are specified.

Modify the makefile to support the `SCHEDULER` macro to compile the specified scheduling algorithm. Use the flags for compilation:- First Come First Serve: FCFS Completely Fair Scheduler: CFS

First Come First Serve [Marks 20]

we will modify the xv6 scheduler from strict round-robin to a firstcome-first-server (FCFS) scheduler. This will involve using the creation time entry in the process control block that was added in part A. We will modify the scheduler function (kernel/proc.c). Scheduler will first have to find a RUNNABLE process with the earliest creation time. The process with the earliest arrival time is the process with the highest priority and therefore the process that is selected for execution. Only when the currently RUNNING process terminates is another process selected to RUN. It is suggested that you comment out the original round-robin scheduler code before adding your new version of the scheduler.

Your compilation process should look something like this: `make clean; make qemu SCHEDULER=FCFS`. Hints:

Use pre-processor directives to declare the alternate scheduling policy in `scheduler()` in `kernel/proc.h`. Edit `struct proc` in `kernel/proc.h` to add information about a process. Modify the `allocproc()` function to set up values when the process starts (see `kernel/proc.h`.)

NOTE `procdump`: This will be useful for debugging (refer `kernel/proc.c`). It prints a list of processes to the console when a user types `Ctrl-P` on the console. You can modify this functionality to print the state of the running process and display the other relevant information on the console.

Use the `procdump` function to print the current status of the processes and check whether the processes are scheduled according to your logic. You are free to do any additions to the given file, to test your scheduler.

You need to implement a simplified version of the Completely Fair Scheduler (CFS) in xv6. The CFS is designed to provide fair CPU time distribution among processes.

Background

The default xv6 scheduler uses a simple round-robin algorithm. Your task is to replace it with a simplified CFS that maintains fairness by tracking how much CPU time each process has received.

Your compilation process should look something like this: `make clean; make qemu SCHEDULER=CFS`.

Requirements

B.1: PRIORITY SUPPORT [MARKS 10]

- 1 Add a nice value for each process.
- 2 Calculate the system weight based on nice value:

Nice 0: weight = 1024

Nice -20: weight = 88761 (highest priority)

Nice 19: weight = 15 (lowest priority)

Use this approximation: $\text{weight} = 1024 / (1.25 ^ \text{nice})$

B.2: VIRTUAL RUNTIME TRACKING [MARKS 20]

- 1 Track the **virtual runtime** of each process. A processes virtual runtime represents the amount of CPU time a process has consumed, normalized by the system weight.
- 2 Initialize `vruntime` to 0 when a process is created.
- 3 Update `vruntime` by the number of ticks the process runs during each time slice.

B.2: SCHEDULING [MARKS 50]

- 1 Maintain processes in order of their `vruntime` (ascending order).
- 2 Always schedule the runnable process with the smallest `vruntime` .
- 3 When a process becomes runnable, insert it in the correct position based on `vruntime` .

B.3: TIME SLICE CALCULATION [MARKS 20]

- 1 Define a target latency of 48 ticks.
- 2 Calculate a time slice as: $\text{time_slice} = \text{target_latency} / \text{number_of_runnable_processes}$
- 3 Enforce a minimum time slice of 3 ticks.
- 4 Each process runs for its calculated time slice before being preempted.

Report [Marks 20]

In your implementation, add logging to print the vRuntime of all runnable processes before every scheduling decision. The log should clearly indicate:

Process ID (PID) of each runnable process.

vRuntime value for each process at that moment.

Which process is selected by the scheduler (should be the one with the lowest vRuntime).

Your log output should allow us to verify that:

The process with the smallest vRuntime is being chosen by the scheduler.

vRuntime values are updating correctly after each time slice.

[Scheduler Tick]

PID: 3 | vRuntime: 200


```
PID: 4 | vRuntime: 150
```

```
PID: 5 | vRuntime: 180
```

```
--> Scheduling PID 4 (lowest vRuntime)
```

The report also must contain brief explanation about the implementation of the specifications. A few lines about your changes for each spec is fine. Include the performance comparison between the default(Round Robin), FCFS and CFS scheduling policies by showing the average waiting and running times for processes. Set the processes to run on only 1 CPU for this purpose. Use the [schedulertest](#) command to get this information.

Bonus: Simplified Preemptive MLFQ Scheduler for XV6 (25 Marks)

Important : This bonus counts towards the overall bonus for the course not particular to this assignment.

Queues & Priorities

FOUR PRIORITY QUEUES: 0 (HIGHEST) → 3 (LOWEST)

Time slices

Queue 0 → 1 tick

Queue 1 → 4 ticks

Queue 2 → 8 ticks

Queue 3 → 16 ticks

Scheduling Rules

New Processes: Start in queue 0 (end of queue).

Priority Selection: Always schedule from the highest non-empty queue. If a process is running from a lower queue and a process arrives in a higher queue, preempt the current one at the next tick.

Time Slice Expiry: If a process uses its full time slice, move it to the end of the next lower queue (unless already in queue 3, then keep it there).

Voluntary Yield (I/O bound): On yielding before its slice ends, re-enter at the end of the

same queue when ready.

Lowest Priority Queue: Use round-robin scheduling.

Starvation Prevention: Every 48 ticks, move all processes to queue 0 to prevent starvation.

Completed processes should leave the system.

Report: Also if you've attempted this section then in the previously mentioned report add comparison for MLFQ as well.

Notes :

"Tick" refers to the clock interrupt timer in kernel/trap.c.

Preemption can only occur at tick boundaries.

```
mini-project1/
├─ shell/
│   ├── src/
│   ├── include/
│   └─ Makefile
├─ networking/
│   ├── client.c
│   ├── server.c
│   ├── sham.h      # Or other shared headers
│   └─ Makefile
├─ xv6/
│   ├── xv6_modifications.patch
│   ├── readcount.c
│   └─ report.md    # Or report.pdf
├─ llm_completions/ # (If you used an LLM for any part)
│   ├── shell_prompt_1.png
│   ├── networking_q1.png
│   └─ ...
└─ README.md
```

